

SMART AIR QUALITY MONITORING NODE (ESP32)

Low-Power Environmental Sensor Node for Off-Grid Deployment

1. Introduction

In many rural and semi-urban areas, air pollution often goes unnoticed due to lack of monitoring systems. This project focuses on building a simple, low-cost air quality monitoring node that can detect harmful gases and alert users in real time.

The system is designed with the idea that it should work in resource-constrained environments, where power and connectivity are limited. Instead of relying heavily on cloud processing, the device performs local decision-making and only sends alerts when necessary.

2. Objective

The main goal of this project is to:

- Measure air quality using sensors
- Process the data locally on the microcontroller
- Trigger alerts when pollution exceeds safe limits
- Minimize unnecessary communication to save power

PART 1 — Firmware Architecture & Power Strategy

MCU Selection and Justification

For this system, the ESP32 microcontroller is selected due to its built-in WiFi capability, sufficient processing power, and ease of development. It allows direct integration with cloud platforms without requiring additional communication modules, which simplifies the overall design.

However, it is important to recognize that the ESP32 is not the most power-efficient option compared to microcontrollers like STM32L0 or nRF52. The choice here is a trade-off between development simplicity and power efficiency. For a prototype and rapid implementation, ESP32 is practical, while a production version should consider a lower-power MCU.

Sleep/Wake Strategy

The firmware follows a duty-cycled operation to minimize energy consumption. The node remains in deep sleep for most of the time and wakes up periodically to perform sensing and processing.

The system wakes up every 5 minutes using a timer-based interrupt, performs sensor readings and decision-making within a short active window of approximately 2–3 seconds, and then returns to deep sleep. This ensures that the system is active for less than 1% of the total time, significantly reducing average power consumption.

Peripheral Power-Gating Plan

Power management is implemented by controlling when different components are active. The ESP32 enters deep sleep whenever processing is complete. The WiFi module is enabled only during data transmission and remains off during idle periods.

However, the MQ135 sensor cannot be fully power-gated because it requires continuous heating for accurate readings. This becomes the primary limitation in achieving ultra-low power operation. Other peripherals, such as the DHT11 sensor, are only active during measurement cycles.

Estimated Average Current Consumption

During active operation, the ESP32 consumes approximately 80–120 mA, mainly due to WiFi usage. In deep sleep mode, the current drops to around 10–20 μ A.

Considering the duty cycle (active for ~3 seconds every 300 seconds), the estimated average current is approximately 1 mA. This value includes both active and sleep phases and represents a realistic operating condition.

Battery and Solar Life Estimation

Assuming the system is powered by a 2000 mAh battery, the estimated operational time can be calculated as:

Battery life $\approx 2000 \text{ mAh} / 1 \text{ mA} \approx 2000 \text{ hours}$ (~80 days)

With the addition of a small solar panel, the system can achieve continuous operation, as the average power requirement is relatively low compared to charging capability.

Memory Footprint Estimate

The firmware is designed to be lightweight and efficient. The estimated memory usage is:

- Flash memory: approximately 150–250 KB
- RAM usage: approximately 15–30 KB

Memory usage is minimized by avoiding complex algorithms, dynamic memory allocation, and heavy libraries. The system relies on simple threshold-based logic, which keeps both RAM and Flash usage under control.

PART 2 — Sensor Reading & Local Decision Logic

Sensor Interface

The system uses the MQ135 air quality sensor connected via the ADC interface of the ESP32 (GPIO 34) to read analog gas concentration levels. Additionally, a DHT11 sensor is connected through a digital GPIO pin to measure temperature. During initialization, the ADC pin is configured for input, and the DHT sensor is initialized using its library to enable periodic readings. This setup ensures that all sensing operations are handled directly by the microcontroller without external dependencies.

Sampling Strategy

To improve reliability, the system does not rely on a single sensor reading. Instead, it collects multiple samples (**typically 5 readings**) at short intervals and computes an average value. This approach helps reduce noise and fluctuations inherent in gas sensors like MQ135. Sampling is performed once per wake cycle (every few minutes), which balances responsiveness with power efficiency. No complex filtering is used, but simple averaging provides sufficient stability for this application.

Local Decision Logic

All decision-making is performed locally on the microcontroller using a threshold-based rule system. The averaged air quality value is compared against predefined thresholds to classify the environment into normal, warning, or critical conditions. To avoid false alerts due to temporary spikes, the system can implement a basic stability check by confirming that the threshold is exceeded for multiple consecutive readings before triggering an alert. This ensures that alerts are meaningful and reduces unnecessary transmissions

Pseudocode:

```
#define BLYNK_TEMPLATE_ID "TMPL3SoZyn_G0"
```

```
#define BLYNK_TEMPLATE_NAME "Air Quality Monitoring"
```

```
#define BLYNK_AUTH_TOKEN "buAMDQEOw4hbl1PDT7x4d4Z-AOR3CRKL"// ⚠ Add this from Blynk
```

```
#include <WiFi.h>
```

```
#include <BlynkSimpleEsp32.h>
```

```
#include "DHT.h"
```

```
// WiFi credentials
```

```
char ssid[] = "project1";
```

```
char pass[] = "123456789";
```

// Pins

#define MQ135_PIN 34

#define LED_PIN 2

#define BUZZER_PIN 15

#define DHTPIN 4

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// Thresholds

#define WARNING_LEVEL 400

#define CRITICAL_LEVEL 600

BlynkTimer timer;

// Read Air Quality

int readAirQuality() {

int sum = 0;

for (int i = 0; i < 5; i++) {

sum += analogRead(MQ135_PIN);

delay(100);

}

return sum / 5;

}

// Send Data to Blynk

```
void sendData() {  
  
    int airValue = readAirQuality();  
  
    float temperature = dht.readTemperature();  
  
  
    Serial.print("Air Quality: ");  
  
    Serial.println(airValue);  
  
  
  
    Serial.print("Temperature: ");  
  
    Serial.println(temperature);  
  
  
  
  
    // Send to Blynk  
  
    Blynk.virtualWrite(V0, airValue);  
  
    Blynk.virtualWrite(V1, temperature);  
  
  
  
  
    // Alert Logic  
  
    if (airValue > CRITICAL_LEVEL) {  
  
        digitalWrite(LED_PIN, HIGH);  
  
        digitalWrite(BUZZER_PIN, HIGH);  
  
        Blynk.logEvent("airalert", " 🚨 Critical Air Pollution!");  
  
    } else if (airValue > WARNING_LEVEL) {  
  
        digitalWrite(LED_PIN, HIGH);  
  
        digitalWrite(BUZZER_PIN, LOW);  
  
        Blynk.logEvent("airalert", " ⚠️ Air Quality Warning!");  
  
    } else {  
  
        digitalWrite(LED_PIN, LOW);  
  
        digitalWrite(BUZZER_PIN, LOW);  
  
    }  
  
}
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    pinMode(LED_PIN, OUTPUT);
```

```
    pinMode(BUZZER_PIN, OUTPUT);
```

```
    dht.begin();
```

```
    // Connect to Blynk
```

```
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
```

```
    // Send data every 10 seconds
```

```
    timer.setInterval(10000L, sendData);
```

```
}
```

```
void loop() {
```

```
    Blynk.run();
```

```
    timer.run();
```

```
}
```

Fault Handling and Robustness

The system includes basic fault handling to ensure reliability. If the sensor returns invalid or out-of-range values, those readings are ignored, and the system continues operation without interruption. In case of a disconnected or malfunctioning sensor, the system avoids triggering false alerts and can optionally send a default warning indicating sensor failure. This prevents incorrect data from affecting decision-making and ensures the node remains operational even under partial failure conditions.

PART 3 — Communication & Data Framing

Protocol Selection and Rationale

For this prototype, WiFi (ESP32 built-in) is used as the communication method due to its ease of implementation and availability of existing infrastructure. It allows seamless integration with platforms like Blynk for real-time monitoring and alerts.

However, from a deployment perspective in off-grid environments, **LoRa is the more appropriate choice** due to its long-range capability (several kilometers), very low power consumption, and minimal infrastructure requirements. Unlike WiFi, LoRa operates effectively in rural areas without continuous internet access.

Design Trade-off:

- WiFi → Easy development, higher power consumption, requires internet
 - LoRa → Low power, long range, better for real-world deployment
-

Payload Format

To reduce transmission overhead and conserve energy, the system uses a compact payload structure.

Example ASCII Frame:

[NodeID]|[AirValue]|[AlertType]

Example:

01|620|2

Where:

- 01 → Node ID
- 620 → Air quality value
- 2 → Alert type (0 = Normal, 1 = Warning, 2 = Critical)

This format is simple, human-readable, and lightweight.

In a more optimized system, this can be converted to a binary format to further reduce payload size.

Transmission Trigger Logic

The system follows an event-driven transmission approach rather than continuous data streaming.

- Data is transmitted only when thresholds are exceeded
- Optional periodic updates can be sent at longer intervals for system monitoring

This significantly reduces:

- Power consumption
- Network usage

Retransmission and Reliability Strategy

In environments with unstable connectivity, reliability becomes critical. The system handles this as follows:

- If transmission fails, the system retries during the next cycle
- No continuous retry loop is used to avoid excessive power drain
- In WiFi mode, reconnection attempts are handled automatically by the ESP32

For more advanced implementations (e.g., LoRa), an acknowledgement-based system can be used, where:

- The receiver confirms successful message delivery
- The node retransmits only if acknowledgement is not received

PART 4 — Reliability & Safety Brief

Watchdog Timer Strategy

The system includes a watchdog mechanism that automatically resets the device if it becomes unresponsive. If the firmware gets stuck during sensor reading, data processing, or communication, the watchdog detects the delay and forces a restart. After reboot, the system starts fresh and continues normal operation. Since the system does not depend on long-term stored data, no critical state needs to be preserved, making recovery simple and reliable.

Plain Explanation for Field Technician

This device is designed to run on its own without needing constant attention. It regularly checks air quality and sends an alert only when pollution levels become unsafe. If something goes wrong, such as a temporary glitch or system freeze, the device will automatically restart itself and continue working. You do not need to manually reset it in most cases. If the device stops sending alerts for a long time, checking power supply and sensor connections is usually enough to restore operation.

Most Likely Failure Mode & Mitigation

The most likely failure in this system is sensor drift or inaccurate readings from the MQ135 sensor, especially due to environmental conditions or long-term usage.

To handle this, the system uses multiple readings and averaging **instead** of relying on a single value. It also ignores abnormal or out-of-range values to prevent false alerts. Regular calibration or sensor replacement is recommended for long-term deployment to maintain accuracy.

3. Conclusion

This project demonstrates the design and implementation of a simple yet effective air quality monitoring node using the ESP32. The system is capable of sensing environmental data, making local decisions based on predefined thresholds, and sending alerts only when necessary. This approach reduces unnecessary communication and helps conserve energy.

While the prototype uses readily available components like the MQ135 sensor and WiFi connectivity for ease of development, it also highlights important real-world challenges such as power consumption and sensor limitations. The design intentionally keeps the logic simple and reliable, ensuring that the system can operate independently even without continuous network access.

Overall, this project reflects a balance between practicality and engineering awareness. It shows how low-power strategies like duty cycling, minimal data transmission, and local processing can be applied to build systems suitable for constrained environments. With improvements in sensor selection and communication methods, the system can be adapted for long-term off-grid deployment.

4. Credits

The following resources and libraries were used in the development of this project:

- **ESP32 Datasheet** — Espressif Systems
- **Blynk IoT Platform and Library** — for cloud communication and notifications
- **Adafruit DHT Sensor Library** — for temperature sensing
- **Arduino IDE and ESP32 Board Package** — for firmware development and deployment

